

Hierarchical AR(1) with partial pooling and national covariates.

$$\mu_{\alpha} \sim \mathcal{N}(5, 3)$$

$$\sigma_{\alpha} \sim \text{Exp}(1/2)$$

$$\mu_{\beta} \sim \mathcal{N}(0.5, 0.3)$$

$$\sigma_{\beta} \sim \text{Exp}(1/0.3)$$

$$\gamma_{fed} \sim \mathcal{N}(0, 1)$$

$$\gamma_{gdp} \sim \mathcal{N}(0, 1)$$

$$\sigma \sim \text{Exp}(1)$$

$$\tilde{\alpha}_s \sim \mathcal{N}(0, 1), \quad \alpha_s = \mu_{\alpha} + \sigma_{\alpha} \tilde{\alpha}_s$$

$$\tilde{\beta}_s \sim \mathcal{N}(0, 1), \quad \beta_s = \mu_{\beta} + \sigma_{\beta} \tilde{\beta}_s$$

$$y_{s,t} \mid \alpha_s, \beta_s, \gamma, \sigma \sim \mathcal{N}(\alpha_s + \beta_s y_{s,t-1} + \gamma_{fed} \text{FEDFUNDS}_t + \gamma_{gdp} \text{GDP}_t, \sigma)$$

```
In [4]: library(cmdstanr)
library(dplyr)
library(tidyr)
library(ggplot2)
set.seed(405)

TRAIN_END <- as.Date("2023-12-01")
```

1. Load and prepare data

```
In [5]: bls <- read.csv("../cleaned_data/bls_data.csv") %>%
  mutate(date = as.Date(date))

fed <- read.csv("../cleaned_data/fedfunds_data.csv") %>%
  mutate(date = as.Date(date))

gdp <- read.csv("../cleaned_data/gdp_data.csv") %>%
  mutate(date = as.Date(date))

bls_cov <- bls %>%
  left_join(fed, by = "date") %>%
  left_join(gdp, by = "date")

bls_train <- bls_cov %>%
  filter(date <= TRAIN_END) %>%
  arrange(state, date) %>%
  group_by(state) %>%
  mutate(
    bls_lag = lag(unemployment_rate),
```

```

    state_id = cur_group_id()
  ) %>%
  filter(!is.na(bls_lag)) %>%
  ungroup()

states <- sort(unique(bls_train$state))
S <- length(states)
N <- nrow(bls_train)

print(paste("Number of states:", S))
print(paste("Number of observations:", N))
head(bls_train)

```

```

[1] "Number of states: 50"
[1] "Number of observations: 28750"

```

A tibble: 6 × 9

state	date	population	unemployed_population	unemployment_rate	fed_rate	gdp_
<chr>	<date>	<int>	<int>	<dbl>	<dbl>	
AK	1976-02-01	160551	11426	7.1	4.77	
AK	1976-03-01	160663	11313	7.0	4.84	
AK	1976-04-01	161579	11286	7.0	4.82	
AK	1976-05-01	162885	11395	7.0	5.29	
AK	1976-06-01	164431	11687	7.1	5.48	
AK	1976-07-01	165869	12181	7.3	5.31	

2. Compile and sample (HMC)

In [6]: `mod2 <- cmdstan_model("model2_hierarchical_ar1.stan")`

```

stan_data <- list(
  N = N,
  S = S,
  y = bls_train$unemployment_rate,
  y_lag = bls_train$bls_lag,
  state_id = bls_train$state_id,
  fed_rate = bls_train$fed_rate,
  gdp_growth = bls_train$gdp_growth
)

fit2 <- mod2$sample(
  data = stan_data,
  seed = 405,
  chains = 4,

```

```
parallel_chains = 4,  
iter_warmup = 1000,  
iter_sampling = 1000,  
refresh = 250  
)  
  
fit2$save_object("../results/model2_fit.rds")
```

Running MCMC with 4 parallel chains...

```
Chain 1 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 2 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 3 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 4 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 4 Iteration:   250 / 2000 [ 12%] (Warmup)
Chain 1 Iteration:   250 / 2000 [ 12%] (Warmup)
Chain 2 Iteration:   250 / 2000 [ 12%] (Warmup)
Chain 3 Iteration:   250 / 2000 [ 12%] (Warmup)
Chain 4 Iteration:   500 / 2000 [ 25%] (Warmup)
Chain 1 Iteration:   500 / 2000 [ 25%] (Warmup)
Chain 2 Iteration:   500 / 2000 [ 25%] (Warmup)
Chain 4 Iteration:   750 / 2000 [ 37%] (Warmup)
Chain 3 Iteration:   500 / 2000 [ 25%] (Warmup)
Chain 1 Iteration:   750 / 2000 [ 37%] (Warmup)
Chain 2 Iteration:   750 / 2000 [ 37%] (Warmup)
Chain 4 Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 4 Iteration:  1001 / 2000 [ 50%] (Sampling)
Chain 3 Iteration:   750 / 2000 [ 37%] (Warmup)
Chain 1 Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 1 Iteration:  1001 / 2000 [ 50%] (Sampling)
Chain 2 Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 2 Iteration:  1001 / 2000 [ 50%] (Sampling)
Chain 4 Iteration:  1250 / 2000 [ 62%] (Sampling)
Chain 3 Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 3 Iteration:  1001 / 2000 [ 50%] (Sampling)
Chain 1 Iteration:  1250 / 2000 [ 62%] (Sampling)
Chain 2 Iteration:  1250 / 2000 [ 62%] (Sampling)
Chain 4 Iteration:  1500 / 2000 [ 75%] (Sampling)
Chain 3 Iteration:  1250 / 2000 [ 62%] (Sampling)
Chain 1 Iteration:  1500 / 2000 [ 75%] (Sampling)
Chain 2 Iteration:  1500 / 2000 [ 75%] (Sampling)
Chain 4 Iteration:  1750 / 2000 [ 87%] (Sampling)
Chain 3 Iteration:  1500 / 2000 [ 75%] (Sampling)
Chain 1 Iteration:  1750 / 2000 [ 87%] (Sampling)
Chain 2 Iteration:  1750 / 2000 [ 87%] (Sampling)
Chain 4 Iteration:  2000 / 2000 [100%] (Sampling)
Chain 4 finished in 638.9 seconds.
Chain 1 Iteration:  2000 / 2000 [100%] (Sampling)
Chain 1 finished in 645.6 seconds.
Chain 3 Iteration:  1750 / 2000 [ 87%] (Sampling)
Chain 2 Iteration:  2000 / 2000 [100%] (Sampling)
Chain 2 finished in 666.5 seconds.
Chain 3 Iteration:  2000 / 2000 [100%] (Sampling)
Chain 3 finished in 690.3 seconds.
```

All 4 chains finished successfully.
Mean chain execution time: 660.3 seconds.
Total execution time: 690.6 seconds.

3. Diagnostics

```
In [7]: diag <- fit2$summary(c("mu_alpha", "sigma_alpha", "mu_beta", "sigma_beta",
                             "gamma_fed", "gamma_gdp", "sigma", "alpha", "beta"))
      select(variable, mean, sd, q5, q95, rhat, ess_bulk, ess_tail)

      print(paste("R-hat range: ", paste(round(range(diag$rhat), 4), collapse = ", "), "\n"))
      print(paste("Bulk ESS range: ", paste(round(range(diag$ess_bulk), 4), collapse = ", "), "\n"))
      print(paste("Tail ESS range: ", paste(round(range(diag$ess_tail), 4), collapse = ", "), "\n"))

      diag %>% filter(variable %in% c("mu_alpha", "sigma_alpha", "mu_beta", "sigma_beta",
                                     "gamma_fed", "gamma_gdp", "sigma", "alpha", "beta"))
```

[1] "R-hat range: 0.9994 - 1.0091"

[1] "Bulk ESS range: 578 - 7378"

[1] "Tail ESS range: 917 - 3601"

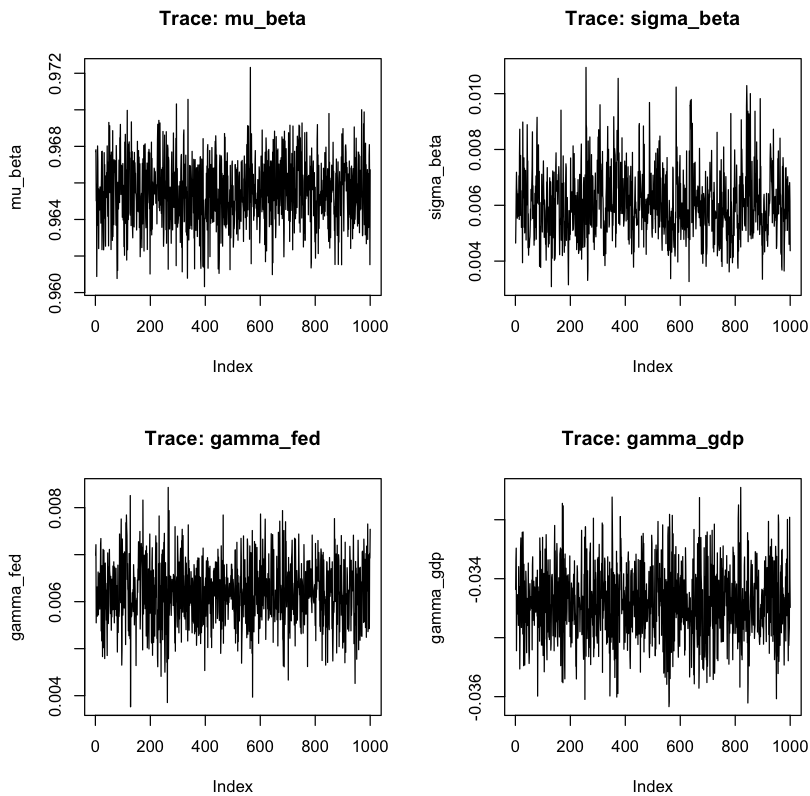
A draws_summary: 7 × 8

variable	mean	sd	q5	q95	rhat	ess.
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
mu_alpha	0.257976918	0.0101672400	0.240850068	0.274317886	1.002771	4242.
sigma_alpha	0.022617735	0.0088301145	0.007325106	0.037425273	1.009127	578.
mu_beta	0.965527966	0.0017829843	0.962540642	0.968449424	1.003563	3603
sigma_beta	0.006074465	0.0012104935	0.004288190	0.008187159	1.001304	1059
gamma_fed	0.006183222	0.0006968717	0.005039520	0.007326301	0.999833	4048.
gamma_gdp	-0.034436660	0.0006483085	-0.035476581	-0.033371557	1.000397	3911.
sigma	0.466272286	0.0019524583	0.462954177	0.469494598	1.004042	6579.

4. Trace plots — hyperparameters

```
In [8]: draws <- fit2$draws(format = "df")

      par(mfrow = c(2, 2))
      plot(draws$mu_beta[draws$.chain == 1], type = "l", main = "Trace: mu_beta")
      plot(draws$sigma_beta[draws$.chain == 1], type = "l", main = "Trace: sigma_beta")
      plot(draws$gamma_fed[draws$.chain == 1], type = "l", main = "Trace: gamma_fed")
      plot(draws$gamma_gdp[draws$.chain == 1], type = "l", main = "Trace: gamma_gdp")
      par(mfrow = c(1, 1))
```

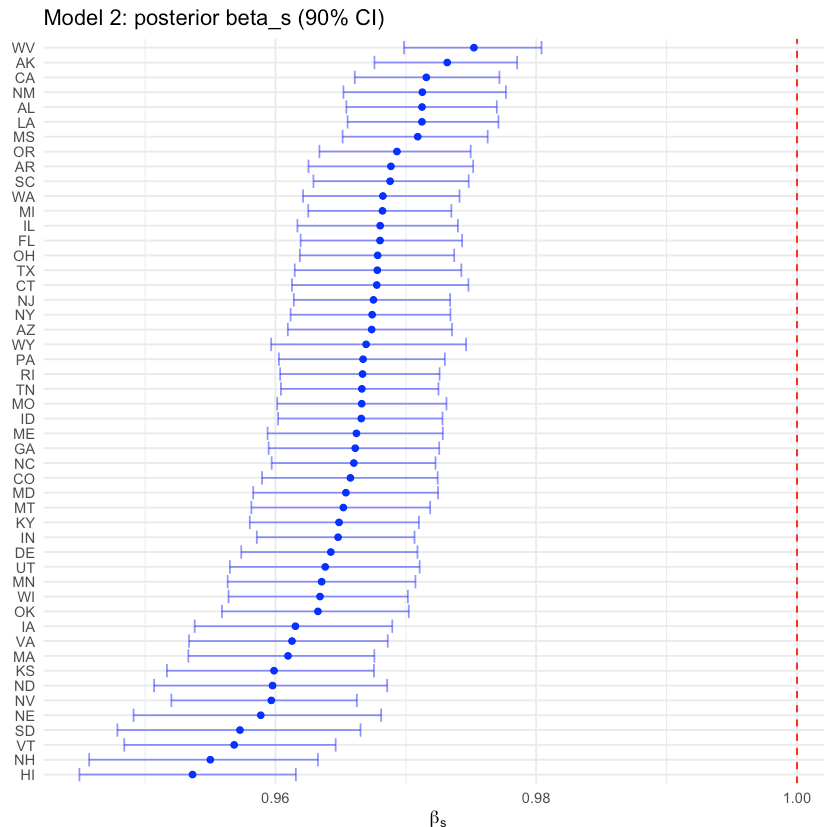


5. Posterior beta_s — all states

```
In [9]: beta_summary2 <- fit2$summary("beta") %>%
  mutate(state = states)

p_beta2 <- ggplot(beta_summary2, aes(x = reorder(state, mean), y = mean)) +
  geom_point(colour = "blue") +
  geom_errorbar(aes(ymin = q5, ymax = q95), colour = "blue", alpha = 0.5) +
  geom_hline(yintercept = 1, linetype = "dashed", colour = "red") +
  labs(title = "Model 2: posterior beta_s (90% CI)", x = NULL, y = expression(beta_s)) +
  coord_flip() +
  theme_minimal()

p_beta2
ggsave("../figures/model2_beta.pdf", p_beta2, width = 6, height = 9)
```



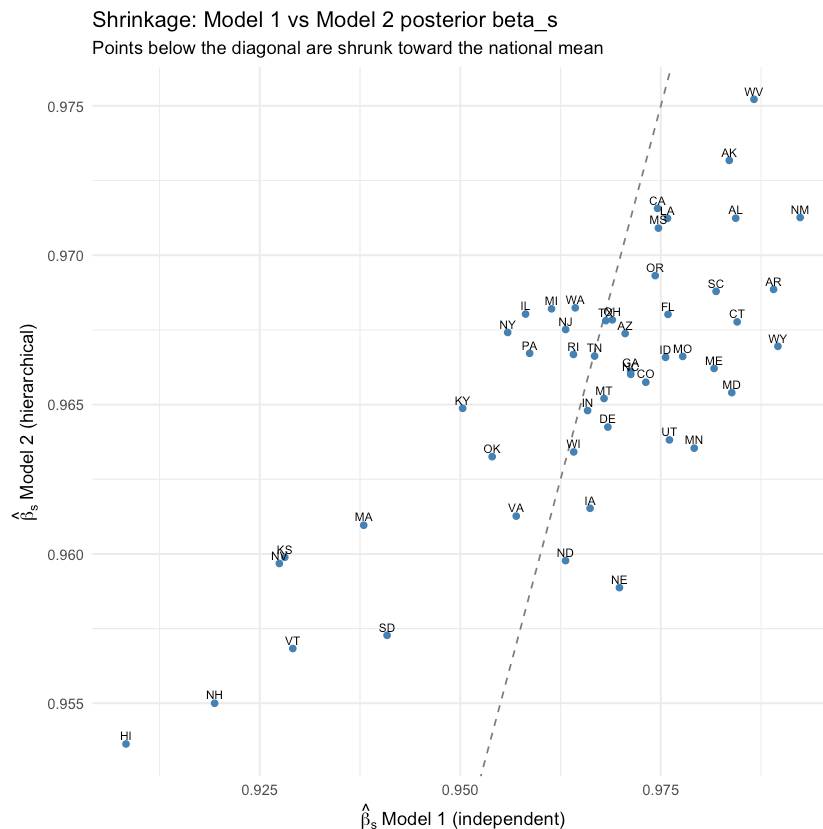
6. Shrinkage plot — Model 1 vs Model 2 β_s

```
In [10]: m1_results <- readRDS("../results/model1_results.rds")

shrinkage_df <- data.frame(
  state = states,
  beta_m1 = m1_results$beta_hat,
  beta_m2 = fit2$summary("beta")$mean
)

p_shrink <- ggplot(shrinkage_df, aes(x = beta_m1, y = beta_m2, label = state)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey50") +
  geom_point(colour = "steelblue") +
  geom_text(size = 2.5, vjust = -0.5) +
  labs(
    title = "Shrinkage: Model 1 vs Model 2 posterior  $\beta_s$ ",
    subtitle = "Points below the diagonal are shrunk toward the national mean",
    x = expression(hat(beta)[s] ~ "Model 1 (independent)"),
    y = expression(hat(beta)[s] ~ "Model 2 (hierarchical)")
  ) +
  theme_minimal()

p_shrink
ggsave("../figures/shrinkage_beta.pdf", p_shrink, width = 7, height = 6)
```



7. Posterior predictive check (PPC)

```
In [11]: y_rep_draws <- fit2$draws("y_rep", format = "matrix")
rep_idx   <- sample(nrow(y_rep_draws), 50)

rep_df <- data.frame(
  value = as.vector(y_rep_draws[rep_idx, ]),
  source = "y_rep"
)
obs_df <- data.frame(
  value = bls_train$unemployment_rate,
  source = "observed"
)

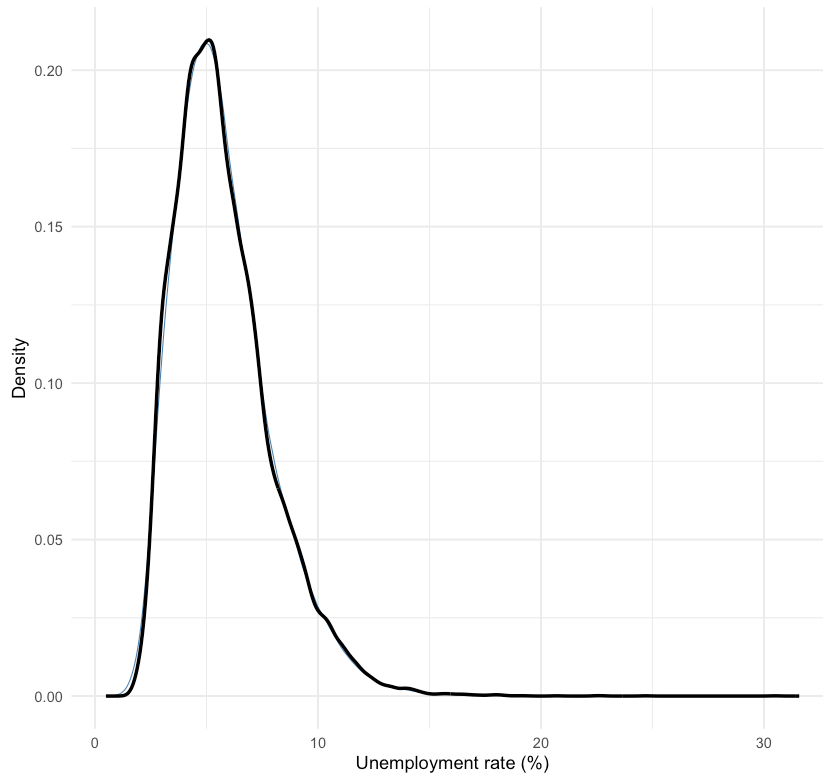
p_ppc2 <- ggplot() +
  geom_density(data = rep_df, aes(x = value, group = source),
    colour = "steelblue", alpha = 0.15, linewidth = 0.3) +
  geom_density(data = obs_df, aes(x = value),
    colour = "black", linewidth = 1) +
  labs(
    title = "Model 2: posterior predictive check",
    subtitle = "Blue = 50 posterior predictive draws, black = observed",
    x = "Unemployment rate (%)", y = "Density"
  ) +
  theme_minimal()

p_ppc2
ggsave("../figures/model2_ppc.pdf", p_ppc2)
```


Saving 7 x 7 in image

Model 2: posterior predictive check

Blue = 50 posterior predictive draws, black = observed



8. Test-set RMSE

```
In [12]: alpha_hat2 <- fit2$summary("alpha")$mean
beta_hat2 <- fit2$summary("beta")$mean
gamma_fed_hat <- fit2$summary("gamma_fed")$mean
gamma_gdp_hat <- fit2$summary("gamma_gdp")$mean

bls_test <- bls_cov %>% filter(date > TRAIN_END) %>% arrange(state, date)
test_dates <- sort(unique(bls_test$date))

y_prev <- bls_cov %>%
  filter(date == TRAIN_END) %>%
  arrange(state) %>%
  pull(unemployment_rate)

forecast_list2 <- vector("list", length(test_dates))

for (i in seq_along(test_dates)) {
  d <- test_dates[i]
  covs <- bls_test %>% filter(date == d) %>% arrange(state)
  y_pred <- alpha_hat2 + beta_hat2 * y_prev +
    gamma_fed_hat * covs$fed_rate +
    gamma_gdp_hat * covs$gdp_growth
  y_actual <- covs$unemployment_rate

  forecast_list2[[i]] <- data.frame(
    state = states,
```

```

    date      = d,
    y_pred    = y_pred,
    y_actual  = y_actual
  )
  y_prev <- y_actual
}

forecasts2 <- bind_rows(forecast_list2)

rmse2 <- forecasts2 %>%
  group_by(state) %>%
  summarise(rmse = sqrt(mean((y_pred - y_actual)^2)), .groups = "drop") %>%
  arrange(desc(rmse))

print("Model 2 overall RMSE:")
print(round(sqrt(mean((forecasts2$y_pred - forecasts2$y_actual)^2)), 3))
print("Worst 5 states:"); print(head(rmse2, 5))
print("Best 5 states:"); print(tail(rmse2, 5))

# RMSE comparison
print("Model 1 overall RMSE:")
print(round(sqrt(mean((bind_rows(readRDS("../results/model1_results.rds"))$fc
                                bind_rows(readRDS("../results/model1_results.rds"))$f

```

```

[1] "Model 2 overall RMSE:"
[1] 0.118
[1] "Worst 5 states:"
[1] 0.118
[1] "Worst 5 states:"
# A tibble: 5 x 2
  state rmse
<chr> <dbl>
1 WV    0.164
2 HI    0.160
3 IN    0.155
4 AL    0.151
5 DE    0.149
[1] "Best 5 states:"
# A tibble: 5 x 2
  state rmse
<chr> <dbl>
1 CT    0.0955
2 MA    0.0944
3 OR    0.0919
4 KS    0.0757
5 NH    0.0727
[1] "Model 1 overall RMSE:"
[1] 0.096

```

9. Forecast plot

```

In [13]: plot_states <- c("CA", "TX", "MI", "NV")

par(mfrow = c(2, 2))

```

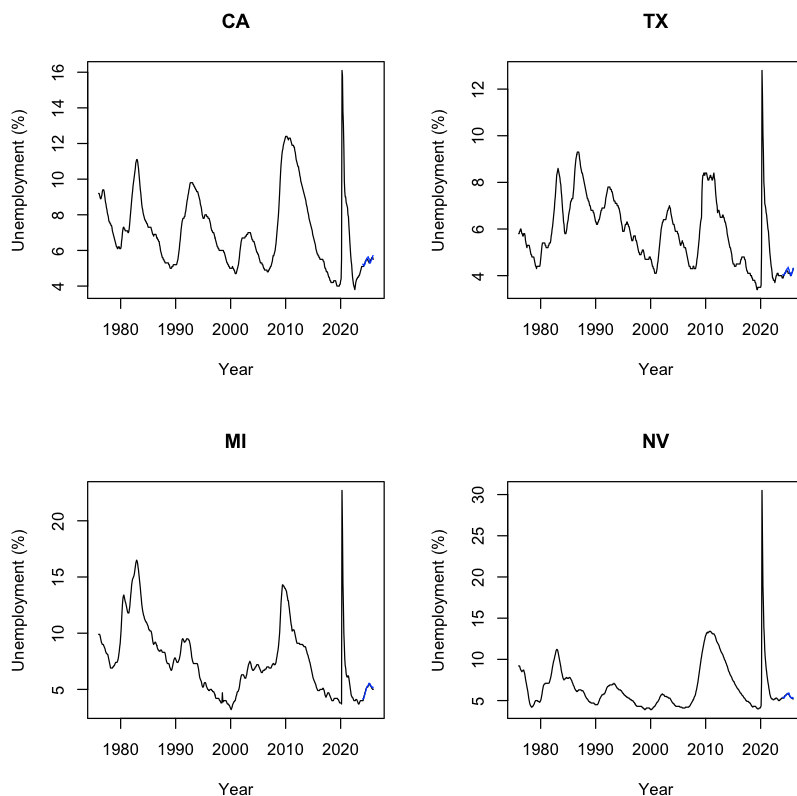
```

for (s in plot_states) {
  obs <- bls %>% filter(state == s) %>% arrange(date)
  pred <- forecasts2 %>% filter(state == s) %>% arrange(date)

  plot(obs$date, obs$unemployment_rate, type = "l",
       main = s, xlab = "Year", ylab = "Unemployment (%)")
  lines(pred$date, pred$y_pred, col = "blue")
}

par(mfrow = c(1, 1))

```



10. VI comparison

```

In [19]: fit2_vi <- mod2$variational(
  data      = stan_data,
  seed      = 405,
  algorithm = "meanfield",
  draws     = 1000
)

# Compare hyperparameter posteriors: HMC vs VI
params <- c("mu_beta", "sigma_beta", "gamma_fed", "gamma_gdp")

hmc_summary <- fit2$summary(params) %>%
  select(variable, mean, q5, q95) %>%
  mutate(method = "HMC")

vi_summary <- fit2_vi$summary(params) %>%
  select(variable, mean, q5, q95) %>%

```

```

mutate(method = "VI")

comparison <- bind_rows(hmc_summary, vi_summary)

ggplot(comparison, aes(x = variable, y = mean,
                      colour = method, shape = method)) +
  geom_point(size = 3, position = position_dodge(0.4)) +
  geom_errorbar(aes(ymin = q5, ymax = q95),
               width = 0.2, position = position_dodge(0.4)) +
  labs(
    title = "HMC vs VI: posterior means and 90% CI",
    subtitle = "Hyperparameters and covariate coefficients",
    x = NULL, y = "Posterior mean"
  ) +
  theme_minimal()

ggsave("../figures/hmc_vs_vi.pdf", width = 7, height = 4)

```

EXPERIMENTAL ALGORITHM:

This procedure has not been thoroughly tested and may be unstable or buggy. The interface is subject to change.

Gradient evaluation took 0.004239 seconds

1000 transitions using 10 leapfrog steps per transition would take 42.39 seconds.

Adjust your expectations accordingly!

Begin eta adaptation.

Iteration: 1 / 250 [0%] (Adaptation)

Iteration: 50 / 250 [20%] (Adaptation)

Iteration: 100 / 250 [40%] (Adaptation)

Iteration: 150 / 250 [60%] (Adaptation)

Success! Found best value [eta = 10] earlier than expected.

Begin stochastic gradient ascent.

iter	ELBO	delta_ELBO_mean	delta_ELBO_med	notes
------	------	-----------------	----------------	-------

Chain 1 stan::variational::normal_meanfield::calc_grad: The number of dropped evaluations has reached its maximum amount (10). Your model may be either severely ill-conditioned or misspecified.

Warning message:

"Fitting finished unexpectedly! Use the \$output() method for more information."
"

Finished in 1.3 seconds.

Error: Fitting failed. Unable to retrieve the draws.

Traceback:

```

1. fit2_vi$summary(params) %>% select(variable, mean, q5, q95) %>%
   . mutate(method = "VI")
2. mutate(., method = "VI")
3. select(., variable, mean, q5, q95)
4. fit2_vi$summary(params)
5. self$draws(variables)
6. stop("Fitting failed. Unable to retrieve the draws.", call. = FALSE)

```

```

In [18]: fit2_vi <- mod2$variational(
  data      = stan_data,
  seed      = 405,
  algorithm = "fullrank",
  draws     = 1000
)

# Compare hyperparameter posteriors: HMC vs VI
params <- c("mu_beta", "sigma_beta", "gamma_fed", "gamma_gdp")

hmc_summary <- fit2$summary(params) %>%
  select(variable, mean, q5, q95) %>%
  mutate(method = "HMC")

vi_summary <- fit2_vi$summary(params) %>%
  select(variable, mean, q5, q95) %>%
  mutate(method = "VI")

comparison <- bind_rows(hmc_summary, vi_summary)

ggplot(comparison, aes(x = variable, y = mean,
                      colour = method, shape = method)) +
  geom_point(size = 3, position = position_dodge(0.4)) +
  geom_errorbar(aes(ymin = q5, ymax = q95),
               width = 0.2, position = position_dodge(0.4)) +
  labs(
    title      = "HMC vs VI: posterior means and 90% CI",
    subtitle   = "Hyperparameters and covariate coefficients",
    x = NULL, y = "Posterior mean"
  ) +
  theme_minimal()

ggsave("../figures/hmc_vs_vi.pdf", width = 7, height = 4)

```

EXPERIMENTAL ALGORITHM:

This procedure has not been thoroughly tested and may be unstable or buggy. The interface is subject to change.

Gradient evaluation took 0.004313 seconds

1000 transitions using 10 leapfrog steps per transition would take 43.13 seconds.

Adjust your expectations accordingly!

Begin eta adaptation.

Iteration: 1 / 250 [0%] (Adaptation)

Iteration: 50 / 250 [20%] (Adaptation)

Iteration: 100 / 250 [40%] (Adaptation)

Iteration: 150 / 250 [60%] (Adaptation)

Iteration: 200 / 250 [80%] (Adaptation)

Iteration: 250 / 250 [100%] (Adaptation)

Success! Found best value [eta = 0.1].

Begin stochastic gradient ascent.

iter	ELBO	delta_ELBO_mean	delta_ELBO_med	notes
100	-699873295.564	1.000	1.000	
200	-2023559660.390	0.827	1.000	
300	-339092522.244	2.207	1.000	
400	-400354521.317	1.694	1.000	
500	-214671333.081	1.528	0.865	
600	-323679504.198	1.329	0.865	
700	-224327793.642	1.203	0.654	
800	-401538438.684	1.108	0.654	
900	-242697612.622	1.057	0.654	
1000	-242244271.871	0.952	0.654	
1100	-268138992.724	0.861	0.443	MAY BE DIVERGIN
G... INSPECT ELBO				
1200	-420282069.853	0.832	0.441	MAY BE DIVERGIN
G... INSPECT ELBO				
1300	-190858714.484	0.456	0.441	
1400	-313818957.011	0.479	0.441	
1500	-121006629.214	0.552	0.441	MAY BE DIVERGIN
G... INSPECT ELBO				
1600	-180250288.802	0.552	0.441	MAY BE DIVERGIN
G... INSPECT ELBO				
1700	-173872760.355	0.511	0.392	MAY BE DIVERGIN
G... INSPECT ELBO				
1800	-128678020.914	0.502	0.362	MAY BE DIVERGIN
G... INSPECT ELBO				
1900	-153901275.610	0.453	0.351	
2000	-111551377.954	0.491	0.362	
2100	-200068362.768	0.525	0.380	MAY BE DIVERGIN
G... INSPECT ELBO				
2200	-79572359.975	0.640	0.392	MAY BE DIVERGIN
G... INSPECT ELBO				
2300	-130827851.471	0.559	0.392	MAY BE DIVERGIN
G... INSPECT ELBO				
2400	-181216892.086	0.548	0.380	MAY BE DIVERGIN
G... INSPECT ELBO				
2500	-76550352.611	0.525	0.380	MAY BE DIVERGIN
G... INSPECT ELBO				
2600	-88638511.708	0.506	0.380	MAY BE DIVERGIN

G... INSPECT ELBO			
2700	-74916455.517	0.521	0.380 MAY BE DIVERGIN
G... INSPECT ELBO			
2800	-74471294.829	0.486	0.380
2900	-66143353.261	0.482	0.380
3000	-71882174.024	0.453	0.278
3100	-87597889.170	0.426	0.183
3200	-58260858.975	0.325	0.183
3300	-56066486.799	0.290	0.179
3400	-50568982.370	0.273	0.136
3500	-87066867.876	0.178	0.136
3600	-67952721.032	0.193	0.179
3700	-98688466.761	0.205	0.179
3800	-50312228.701	0.301	0.281
3900	-56258611.283	0.299	0.281
4000	-43853758.114	0.319	0.283
4100	-76596779.665	0.344	0.311
4200	-56778606.805	0.329	0.311
4300	-52562744.302	0.333	0.311
4400	-26347191.238	0.421	0.349
4500	-30994130.205	0.394	0.311
4600	-44264984.649	0.396	0.311
4700	-32181059.944	0.403	0.349
4800	-27722873.005	0.323	0.300
4900	-33820431.100	0.330	0.300
5000	-41623454.524	0.321	0.300
5100	-25362317.166	0.342	0.300
5200	-23796398.296	0.314	0.187
5300	-21615220.519	0.316	0.187
5400	-19955221.643	0.224	0.180
5500	-22162281.127	0.219	0.180
5600	-23736986.546	0.196	0.161
5700	-25273586.800	0.165	0.101
5800	-21784813.720	0.165	0.101
5900	-23533205.844	0.154	0.100
6000	-19305126.069	0.157	0.100
6100	-15257348.765	0.120	0.100
6200	-27330721.257	0.157	0.101
6300	-21986687.586	0.171	0.160
6400	-14836207.733	0.211	0.219
6500	-22209492.926	0.234	0.243
6600	-16300744.623	0.264	0.265
6700	-11345255.328	0.302	0.332
6800	-10960943.028	0.289	0.332
6900	-12345465.487	0.293	0.332
7000	-15164780.634	0.290	0.332
7100	-14474867.008	0.268	0.332
7200	-28231336.423	0.272	0.332
7300	-16729184.156	0.317	0.362
7400	-9252232.407	0.349	0.362
7500	-8112472.452	0.330	0.362
7600	-9404070.788	0.308	0.186
7700	-8756141.312	0.272	0.140
7800	-6966724.661	0.294	0.186
7900	-8554961.262	0.301	0.186
8000	-7803173.605	0.292	0.186

8100	-10223129.931	0.311	0.237
8200	-8562813.942	0.282	0.194
8300	-7344924.270	0.230	0.186
8400	-8584246.703	0.163	0.166
8500	-7438997.711	0.164	0.166
8600	-6198882.626	0.171	0.186
8700	-5657323.195	0.173	0.186
8800	-6095044.915	0.154	0.166
8900	-5196147.972	0.153	0.166
9000	-5452797.204	0.148	0.166
9100	-5958509.556	0.133	0.154
9200	-6704196.489	0.125	0.144
9300	-5146967.708	0.138	0.144
9400	-5930608.566	0.137	0.132
9500	-5345784.817	0.133	0.111
9600	-4734369.650	0.126	0.111
9700	-4273715.265	0.127	0.111
9800	-3211559.041	0.153	0.129
9900	-5019483.001	0.172	0.129
10000	-3560458.179	0.208	0.132

Informational Message: The maximum number of iterations is reached! The algorithm may not have converged.

This variational approximation is not guaranteed to be meaningful.

Drawing a sample of size 1000 from the approximate posterior...

COMPLETED.

Finished in 42.1 seconds.

HMC vs VI: posterior means and 90% CI

Hyperparameters and covariate coefficients

